

Name : Mayank Dudhatra

UID : 24BTCSE100

Batch : B.Tech - CSE - Sem-4

Subject : DEVOPS ESSENTIALS - 030704405

1. Apply Design Thinking process to a real-life problem: Empathize, Define, Ideate, Prototype, and Test.

Problem Statement: Users find it difficult to track their daily water intake, leading to dehydration and fatigue.

- **Empathize:** Conducted interviews with college students and office workers. Found that most people forget to drink water because they are hyper-focused on work or lack a visual reminder.
 - **Define:** "Busy professionals and students need a non-intrusive way to track and be reminded of hydration throughout the day because their current schedules prioritize screen time over physical health."
 - **Ideate:** Solutions include a smart water bottle, a mobile app with push notifications, or a desktop widget that changes color based on time elapsed since the last drink.
 - **Prototype:** Developed a low-fidelity mobile app wireframe with a simple "One-tap" button to log water intake and a progress bar shaped like a water drop.
 - **Test:** Shared the wireframe with 5 users. Feedback suggested that manual logging is tedious; users requested an automated reminder that syncs with their calendar breaks.
-

2. Compare and demonstrate Agile vs Waterfall through a mock software development cycle.

<u>Feature</u>	<u>Waterfall Model</u>	<u>Agile Model (Scrum)</u>
Approach	Linear and Sequential.	Iterative and Incremental.
Requirements	Fixed at the start.	Evolving and flexible.
Development	One big "Bang" release after months.	Small releases every 2 weeks (Sprints).
Demonstration	You spend 2 months on docs, 3 on code, then test. If the user wants a change in month 4, it is costly.	Sprint 1: Build Login. Sprint 2: Build Grade Input. User sees progress every 14 days and gives feedback.

3. Simulate Scrum framework: Create a Scrum team, conduct sprint planning, and track sprint progress.

Scrum Team:

- **Product Owner:** Manages the Backlog (mayank).
- **Scrum Master:** Removes blockers (Project Mentor).
- **Developers:** 3 Full-stack developers.

Sprint Planning:

- **Goal:** Implement User Authentication.
- **Task Breakdown:** Set up MongoDB schema, Create Login API, Design Frontend UI.

Tracking Progress:

- **Daily Stand-up:** 15-minute meeting to discuss: What did I do yesterday? What will I do today? Are there any blockers?
 - **Tool:** Use a **Kanban Board** with columns: To Do, In Progress, Testing, Done.
-

4. Set up Git and GitHub: Create repositories, practice branching, merging, and pull requests.

- **Initialize Repository:** `git init devops-assignment`
 - **Create Remote:** Create a repo on GitHub and link it: `git remote add origin <URL>`
 - **Branching:** Create a new feature branch: `git checkout -b feature-auth`
 - **Commit:** `git add . -> git commit -m "Added login logic"`
 - **Push:** `git push origin feature-auth`
 - **Pull Request (PR):** Open GitHub, click "Compare & Pull Request," and merge the feature branch into the main branch after review.
-

5. Implement a Git workflow using feature branches and collaborative team commits.

Rule: Never commit directly to `main`.

Workflow Step-by-Step:

1. **Pull Latest:** `git pull origin main` to stay updated.
 2. **Feature Branch:** Create `feature/ui-design`.
 3. **Collaborate:** Multiple team members push to the same feature branch to integrate parts of the UI.
 4. **Conflict Resolution:** If two people edit the same line, Git flags a conflict. We manually resolve it, commit, and then merge.
-

6. Install and configure Jenkins on a local machine or VM for CI setup.

- **Prerequisites:** Install Java (JRE/JDK).
- **Download:** Download the `jenkins.war` file or install via package manager (`apt/brew`).
- **Run:** Execute `java -jar jenkins.war` in the terminal.
- **Setup:** Open `localhost:8080` in a browser.
- **Unlock:** Retrieve the initial admin password from the file path shown in the terminal.

- **Plugins:** Select "Install suggested plugins" to get Git and Pipeline support.
-

7. Create a basic CI pipeline in Jenkins that pulls code from Git and builds the project.

1. Create a **New Item** -> **Pipeline**.
 2. In the Pipeline script section, use the following Groovy code:

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/your-username/your-repo.git'
      }
    }
    stage('Build') {
      steps {
        echo 'Building the project...'
      }
    }
  }
}
```
-

8. Install Docker and run basic containers on a local development environment.

1. **Install:** Download Docker Desktop for Windows/Mac or `sudo apt install docker.io` for Linux.
 2. **Verify:** `docker --version`
 3. **Run Hello World:** `docker run hello-world`
 4. **Run Nginx Server:** `docker run -d -p 8080:80 nginx`
 - This pulls the Nginx image and runs it in detached mode, mapping port 8080 on your machine to port 80 in the container.
-

9. Write a Dockerfile and build a custom Docker image for a simple application (e.g., Node.js or Python).

Example: Node.js App

1. **Create file named Dockerfile:**

```
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
```

CMD ["node", "index.js"]

2. **Build Image:** `docker build -t my-node-app .`
 3. **Run Image:** `docker run -p 3000:3000 my-node-app`
-

10. Use Docker Compose to simulate a multi-container environment (e.g., web app + database).

Create a `docker-compose.yml` file:

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - db
  db:
    image: mongo:latest
    ports:
      - "27017:27017"
```

Command: Run `docker-compose up` to start both the web app and the database simultaneously.

11. Integrate Jenkins and Docker to automate builds and containerization in a CI/CD workflow.

In the Jenkins Pipeline script, add a stage for Docker:

```
stage('Docker Build & Push') {
  steps {
    script {
      sh 'docker build -t mayank-app:latest .'
      sh 'docker tag mayank-app:latest mydockerhub/mayank-app:latest'
      // sh 'docker push mydockerhub/mayank-app:latest'
    }
  }
}
```

This automates the process of creating a new image every time code is pushed to Git.

12. Install and configure Minikube or Docker Desktop with Kubernetes support.

1. **Enable K8s:** In Docker Desktop settings, check "Enable Kubernetes" and restart.
 2. **Verify:** Run `kubectl cluster-info` and `kubectl get nodes`.
 3. **Minikube Alternative:** Install Minikube and run `minikube start`.
-

13. Deploy a containerized app on Kubernetes, define Pods, Deployments, and Services using YAML files.

Create `deployment.yaml`:

```
apiVersion: apps/v1
```

```
kind: Deployment
metadata:
  name: my-app-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

Command: `kubectl apply -f deployment.yaml`

14. Perform rolling updates and scale replicas using Kubernetes commands and configuration.

- **Scaling:** To increase the number of running instances to 5: `kubectl scale deployment my-app-deployment --replicas=5`
 - **Rolling Update:** To update the image to a newer version without downtime: `kubectl set image deployment/my-app-deployment my-app-container=nginx:1.21`
 - **Check Status:** `kubectl rollout status deployment/my-app-deployment`
-